

01.0 Que es Bash

1. ¿Qué es Bash?

Bash es un **intérprete de comandos**, también llamado **shell**.

Ambos términos son sinónimos y se refieren al mismo concepto.

Su función principal es servir como **interfaz entre el usuario y el sistema operativo**:

- El usuario escribe comandos.
- Bash los interpreta.
- Si los comandos son correctos, los envía al **kernel** del sistema para que se ejecuten.

 Importante:

Para que un comando pueda ejecutarse, debe cumplir unas reglas de sintaxis.

Por eso, **Bash es intrínsecamente un lenguaje**, no solo una herramienta.

2. ¿Dónde se usa Bash?

- Es la **shell predeterminada** en la mayoría de las distribuciones Linux.
- También está disponible en:
 - Windows, mediante **WSL (Windows Subsystem for Linux)**
 - Entornos como **Git Bash**

3. Bash como intérprete de comandos

Uno de los modos de uso de Bash es el **modo interactivo**.

Modo interactivo

- Es el Bash que usamos cuando abrimos un **terminal**.
- Acepta comandos escritos por teclado.
- Ejecuta cada comando en el momento en que se pulsa Enter.

Ejemplos típicos:

- Listar archivos
- Moverse entre directorios
- Ejecutar programas
- Encadenar comandos

Este es el uso más habitual para usuarios y administradores de sistemas.

4. Bash como lenguaje de scripting

El segundo modo de uso de Bash es como **lenguaje de programación**, también llamado **lenguaje de scripting**.

¿Qué es un script?

Un **script de Bash** es:

- Un archivo de texto
- Que contiene instrucciones escritas en Bash
- Ejecutadas **de forma secuencial**
- Con el objetivo de **automatizar tareas**

Ejemplos de tareas automatizables:

- Copias de seguridad
- Procesamiento de archivos
- Instalaciones repetitivas
- Limpieza del sistema

5. Dos modos, un mismo Bash

Aunque hablamos de dos modos distintos:

- Bash **interactivo**
 - Bash **como scripting**
- No son cosas separadas.**

Lo que ejecutas como usuario en la terminal es exactamente lo mismo que ejecutas como programador en un script.

No existe una separación tajante entre ambos:

- Son **el mismo lenguaje**
- Usado en **dos niveles diferentes**

¿Puedo ejecutar en la terminal lo que hay en un script?

Sí:

- Puedes ejecutar cada instrucción del script manualmente.
- Incluso varias instrucciones en una sola línea, usando los separadores adecuados.

¿Puedo llevar a un script lo que hago en la terminal?

También sí:

- Tomas los comandos que ejecutas manualmente.

- Los guardas en un archivo, uno detrás de otro.
- Añades algunos elementos extra (que veremos más adelante).
- Eso ya es un script funcional.

01.1 Que es un script en Bash

1. Definición

Un **script en Bash** es un **archivo de texto** que contiene **instrucciones válidas en Bash** y que puede ejecutarse para automatizar tareas.

2. Creación de un archivo

Un script no es más que un fichero. Puede crearse desde el terminal de varias formas:

```
touch script.sh
true > script.sh : > script.sh
cat > script.sh # CTRL + D para terminar
```

También puede crearse con editores de texto como `nano` o `vim`.

3. Condiciones de un script

- Ser un **archivo de texto plano**
- Contener **comandos válidos de Bash**
(comandos normales y elementos del lenguaje)

3.2 Condiciones opcionales - Ojo según el caso

- Permiso de ejecución (x)
- Línea **shebang**, por ejemplo:
`#!/bin/bash`

4. Permisos y shebang según ejecución

Forma de ejecución	Permiso x	Shebang
<code>bash script.sh</code>	No	No
<code>sh script.sh</code>	No	No
<code>./script.sh</code> (o ruta absoluta)	Sí	Sí

 `sh` puede no ser Bash (a menudo es **Dash**).

5. Formas de ejecutar un script en bash

```
bash script.sh
```

```
sh script.sh #no recomendado para Bash avanzado
```

```
./script.sh
```

#Los siguientes - No requieren permisos ni shebang. Se ejecutan en ****la shell actual****, no en un proceso nuevo

```
source script.sh
```

```
. script.sh
```

01.2 Variables

1. Qué es una variable

Una **variable** es un nombre que almacena un valor.

En Bash, si no se indica lo contrario, **el valor es una cadena de texto (string)**.

Una variable puede tener **atributos** adicionales, que veremos más adelante.

Todo lo explicado aquí en modo interactivo es **directamente aplicable a scripts**.

2. Asignación de variables

```
var= # string vacío var=mistring var="mi string"
```

 **Regla clave:**

En la asignación **no debe haber espacios alrededor del =**.

Incorrecto (Dará un error "orden no encontrada"):

```
var = valor
```

3. Reglas de nombres de variables

- Deben empezar por **letra** o **guion bajo (_)**
- Solo pueden contener:
 - Letras
 - Números
 - Guiones bajos (_)
- Son **case-sensitive** (`var` $\neq$ `Var`)
- **No permitidos:**
 - Espacios
 - Guion medio (-)
 - Símbolos especiales (`! @ # $ % ^ & * ( )`)

4. Acceso al valor de una variable

Para acceder al valor se usa el símbolo `$`.

```
var=1234
echo $var
```

Con comillas dobles:

```
echo "Variable var contiene $var"
(Salida)> Variable var contiene 1234
```

Con comillas simples (no expande variables):

```
echo 'Variable var contiene $var'
(Salida)> Variable var contiene $var
```

5. Uso de `${var}`

```
echo ${var}
```

Se usa especialmente cuando:

- La variable está junto a otros caracteres
- Se quiere evitar ambigüedad

```
echo "${var}123" `
```

También permite otras operaciones que se verán más adelante.

6. `declare` / `typeset`

Bash es un lenguaje **no tipado**, pero permite asignar **atributos** a las variables mediante los builtins `declare` o `typeset`.

```
declare variable
```

Con `declare`:

- Se pueden definir uno o varios atributos
- Bash adapta su comportamiento según esos atributos
- Es una forma de “tipar” variables

Para ver opciones:

```
help declare
```

7. Concatenación de variables

Las variables se concatenan **directamente**, una a continuación de otra.

```
var="$var1$var2"
```

⚠ Cuidado con los espacios: Bash no los añade automáticamente.

01.3 Parámetros Posicionales

1. Paso de variables externas a un script

Es posible pasar valores a un script antes de ejecutarlo.

En Bash se llaman **parámetros posicionales** porque se accede a ellos por su posición.

Dentro de un script se referencian como:

```
$1  
$2  
$3  
$n
```

2. Ejecución con parámetros

Ejemplo de ejecución de un script con argumentos:

```
bash 1.3_Parametros_Posicionales.sh Juan 22 300
```

Correspondencia dentro del script:

```
$1 #Juan  
$2 #22  
$3 #300
```

3. Parámetros con espacios: uso de comillas

Si un parámetro contiene espacios, debe ir entre comillas:

```
bash 1.3_Parametros_Posicionales.sh "Juan Antonio" 22 300
```

También es válido:

```
bash 1.3_Parametros_Posicionales.sh 'Juan Antonio' 22 300
```

4. Comillas simples vs dobles

Las comillas simples son restrictivas.

Las comillas dobles permiten expansión.

Definición de variable:

```
name="Juan Antonio"
```

Paso de parámetros:

```
bash 1.3_Parametros_Posicionales.sh '$name' 22 300
```

```
bash 1.3_Parametros_Posicionales.sh "$name" 22 300
```

Resultado:

- Comillas simples → se pasa el texto literal (En el ejemplo anterior `$name`)
- Comillas dobles → se pasa el valor de la variable (En el ejemplo anterior `Juan Antonio`)

Esto es así porque las comillas dobles admiten expansión de variables.

5. Restricción de los parámetros posicionales

Los parámetros posicionales no pueden reasignarse directamente.

Esto produce error:

```
$1="John"
```

Para modificarlos deben usarse los builtins adecuados.

6. Builtin set

El builtin `set` permite redefinir los parámetros posicionales:

```
set $1 "John"
```

7. Builtin shift

El builtin `shift` desplaza los parámetros:

```
shift
```

Efecto:

- Se elimina:

```
$1
```

- El resto se desplaza una posición a la izquierda

Es útil para recorrer argumentos sin conocer cuántos hay.

8. Parámetros posicionales en funciones

Las funciones usan parámetros posicionales igual que los scripts:

```
$1
```

```
$2
```

```
$3
```

La diferencia es el origen de los valores:

- Script → línea de comandos
- Función → llamada a la función

9. Parámetros con más de un dígito

Esto NO funciona como se espera:

```
echo $10 #escribirá el valor de $1 y concatena el caracter "0"
```

Forma correcta:

```
echo ${10} #escribe el valor del parámetro 10
```

Cuando el índice tiene más de un dígito, es obligatorio usar llaves.

01.4 Variables Especiales

En Bash existen **variables especiales o reservadas** propias del lenguaje. Todas comienzan por el símbolo `$` y tienen un significado específico.

Pueden usarse tanto en la shell interactiva como en scripts.

A continuación se presentan las más relevantes para scripting.

`$#`

Devuelve el **número de parámetros posicionales** pasados al script o función.

```
echo "El número de parámetros es: "$#
```

Se usa para:

- Validar el número de argumentos
- Comprobar si el script ha recibido los parámetros esperados

`$?`

Contiene el **estado de salida del último comando ejecutado**.

```
echo $?  
  
if [[ $? != 0 ]]; then ...
```

Valores:

- `0` → ejecución correcta
- distinto de `0` → error

Se usa para:

- Detectar fallos
- Tomar decisiones según el resultado de un comando
- Devolver el estado correcto al proceso padre

`$$`

Devuelve el **PID del proceso actual**.

```
echo $$
```

```
kill -9 $$
```

Comportamiento:

- En la shell → PID de la shell actual
- En scripts:
 - Ejecutado con `bash` o `./` → PID del script
 - Ejecutado con `source` → PID de la shell padre

\$0

Contiene el **nombre del script** que se está ejecutando.

```
echo $0
```

Uso típico:

- Mostrar mensajes de ayuda
- Referenciar el propio script durante la ejecución

⚠ En una subshell puede contener el nombre del proceso padre.

\$* y \$@

Ambos representan **todos los parámetros posicionales**, pero se comportan distinto al entrecomillarse.

\$*

```
$*
```

- Une todos los parámetros en **una única cadena**
- Pierde la separación original entre argumentos (Cuidado espacios)

\$@

```
$@
```

- Mantiene **cada parámetro por separado**
- Respeta los argumentos con espacios

Conclusión:

En scripts, **usa siempre** `$@` para preservar correctamente los parámetros.

¿Para qué se usan estas variables en scripts?

Manejo de parámetros

```
 $#  
 $*  
 $@
```

Permiten:

- Recorrer parámetros
- Validar cantidad
- Procesar argumentos correctamente

Información del script

```
 $0
```

Permite obtener el nombre del script en ejecución.

Control de errores

```
 $?
```

Permite:

- Detectar fallos
- Actuar en consecuencia
- Devolver el estado correcto al proceso padre

Gestión de procesos

```
 $$
```

Permite:

- Obtener información del proceso actual
- Identificar la shell o el script en ejecución

Resumen

Puntos importantes:

- `$#` → número de parámetros
- `$?` → estado del último comando
- `$$` → PID del proceso
- `$0` → nombre del script
- `$*` → todos los parámetros como un solo string
- `@` → todos los parámetros separados (recomendado)

01.5 Redirecciones simples

En Bash, una **redirección** consiste en indicar:

- De dónde leer los datos
- A dónde enviar los resultados

En lugar de usar siempre:

- Teclado → entrada estándar
- Pantalla → salida estándar

Podemos redirigir la entrada y salida hacia **archivos u otros streams**.

Canales estándar en Bash

Bash trabaja con tres canales básicos:

```
0
1
2
```

Significado:

- 0 → stdin (entrada estándar)
- 1 → stdout (salida estándar)
- 2 → stderr (error estándar)

Salida estándar (stdout) → archivo

Por defecto, los programas escriben en la pantalla.

Con `>` redirigimos la salida a un archivo (crea o sobrescribe):

```
echo "Hola mundo" > salida.txt
```

Para **añadir al final** sin borrar el contenido previo:

```
echo "Otra línea" >> salida.txt
```

Entrada estándar (stdin) ← archivo

Por defecto, los programas leen del teclado.

Con `<` redirigimos la entrada desde un archivo:

```
wc -l < salida.txt
```

El comando:

- Lee desde `stdin`
- Cuenta las líneas
- No necesita que se indique el nombre del archivo como argumento

Error estándar (stderr) → archivo

Los errores no van por la salida estándar, sino por un canal independiente.

Para redirigir errores:

```
ls carpeta_que_no_existe 2> errores.txt
```

Resultado:

- El error se guarda en `errores.txt`
- No se muestra en pantalla

Redirecciones explícitas por canal

Salida estándar explícita:

```
echo "Linea 2" 1> /tmp/output.txt
```

Podemos omitir el 1, bash toma por defecto la salida estándar si no se especifica número

```
echo "Linea 2" > /tmp/output.txt
```

Entrada estándar explícita:

```
wc -w 0< /etc/passwd
```

Podemos omitir el 0, bash toma por defecto la entrada estándar si no se especifica número

```
wc -w < /etc/passwd
```

Redirigir stdout y stderr juntos

Para guardar **toda la salida**, tanto normal como errores:

```
comando > todo.txt 2>&1
```

Esto redirige:

- `stdout` → archivo
- `stderr` → mismo destino que `stdout`

Redirecciones y scripts

Es posible pasar datos a un script usando entrada estándar:

```
./script.sh < fichero.txt
```

Dentro del script, la entrada puede leerse desde:

```
cat /dev/stdin
```

Si se necesita procesar línea a línea, se usan bucles (se verá más adelante).

Diferencia entre parámetro y redirección

Pasar un archivo como parámetro

- Se recibe el **nombre del archivo**
- Se conoce su ruta en el sistema
- Se accede al fichero directamente

Pasar datos por redirección

- Se recibe un **stream de datos** (El contenido)
- No se sabe si proviene de un archivo, un comando u otra fuente
- Solo se procesa la información entrante

01.6 Tuberías

Tuberías (pipes) en Bash

Una **tubería** o **pipe** (|) es un mecanismo para **conectar comandos** formando una cadena de ejecución.

La salida estándar de un comando se convierte en la entrada estándar del siguiente.

En una secuencia:

- Cada comando **lee la salida del comando anterior**
- No es necesario usar archivos intermedios

Funcionamiento básico

Entre dos comandos `cmd1` y `cmd2`:

```
cmd1 | cmd2
```

- `cmd1` escribe en stdout
- `cmd2` lee desde stdin

Ejemplos simples en la shell

Contar el número de archivos de un directorio:

```
ls -l | wc -l
```

Buscar líneas con una palabra, ordenar y eliminar duplicados:

```
cat file.txt | grep "error" | sort | uniq
```

Uso de tuberías en scripts

Ejemplo de script que analiza un archivo recibido como parámetro:

```
echo "Las 5 palabras más frecuentes:"  
tr -s ' ' '\n' < "$archivo" \  
| tr -d '[:punct:]' \  
| tr '[:upper:]' '[:lower:]' \  
| sort \  
|
```

```
| uniq -c \  
| sort -nr \  
| head -5
```

Tuberías y errores (stdout + stderr)

Por defecto, una tubería solo conecta **stdout**.

Si queremos incluir también **stderr**, usamos:

```
|&
```

Esto es equivalente a:

```
comando 2>&1 | otro_comando
```

Ejemplo con errores incluidos

```
( ls /etc/passwd; ls /noexiste; ls /bin/bash; ls /tampoco ) |& grep -c "No  
such"
```

En este caso:

- Se combinan salida normal y errores
- `grep` procesa ambos flujos

01.7 Entorno de ejecución

Como cualquier programa, la **shell de Bash** se ejecuta dentro de un **entorno de ejecución**.

Ese entorno es un **conjunto de variables** (pares nombre=valor) y, en algunos casos, **funciones exportadas**, que están disponibles para los procesos que se ejecutan desde esa shell.

Ver las variables del entorno

Para listar las variables de entorno actuales:

```
env
```

o bien:

```
printenv
```

Variables de entorno habituales

Algunas variables que suelen estar siempre presentes:

- `PATH` → directorios donde se buscan los programas
- `HOME` → directorio personal del usuario
- `USER` → nombre del usuario actual
- `PWD` → directorio de trabajo actual

Estas variables se definen automáticamente al iniciar la shell, a partir de ficheros como:

- `~/.bashrc`
- `~/.profile`
- `/etc/profile`

Variables locales vs variables de entorno

No todas las variables que definimos pasan a formar parte del entorno exportado.

Ejemplo:

```
VAR1="var local"  
export VAR2="var global"
```

Comportamiento:

- VAR1 es **local a la shell**
- VAR2 es una **variable de entorno**

Consecuencias:

- VAR1 no aparece en `env` , solo en `set`
- VAR2 aparece en `env`
- VAR2 será heredada por cualquier programa o script ejecutado desde esta shell

Herencia del entorno en scripts

Cuando ejecutamos un script con:

```
bash script.sh
```

o:

```
./script.sh
```

El script:

- Recibe **una copia del entorno** del proceso padre
- Puede usar variables exportadas (`PATH` , `HOME` , `VAR2` , etc.)
- Puede modificar su propio entorno o el de sus subprocesos
- **No puede modificar el entorno de la shell que lo lanzó**

Al finalizar el script, cualquier cambio en el entorno se pierde.

Caso especial: source

Si ejecutamos un script con:

```
source script.sh
```

o:

```
. script.sh
```

Entonces:

- No se crea un proceso hijo
- El script se ejecuta **en la shell actual**

- Las variables definidas o modificadas **persisten** tras finalizar el script

Esto permite:

- Cargar configuraciones
- Modificar el entorno de la shell activa
- Exportar variables de forma permanente en la sesión

01.8 Exit Status

En Bash, **todo comando devuelve un estado de salida** (*exit status*) cuando termina su ejecución.

Este valor indica si el comando:

- Se ejecutó correctamente
- Falló por algún motivo

Estado de salida y la variable \$?

Cuando un comando termina, Bash guarda su estado de salida en la variable especial:

```
$?
```

Convención general:

- 0 → éxito
- distinto de 0 → error

El rango de valores va de 0 a 255 .

Ejemplo básico en la shell

```
ls -l /home/sec2john
```

```
echo $?
```

Resultado:

- Devuelve 0 si el comando tuvo éxito

```
ls -l /home/no_existe
```

```
echo $?
```

Resultado:

- Devuelve un valor distinto de 0

Significado de los valores distintos de cero

Cada comando define **su propia semántica** para los valores de error.

Esto significa:

- No todos los comandos interpretan igual los valores `1` , `2` , etc.
- El significado depende del propio comando

Ejemplos:

- `ls`
- `make`

Puedes consultar el significado exacto en su documentación:

```
man ls
```

```
man make
```

¿Para qué sirve el exit status?

Principalmente para:

- Controlar el flujo de un script
- Comprobar si un comando anterior falló
- Tomar decisiones en estructuras condicionales
- Encadenar comandos de forma lógica

Uso en encadenamiento de comandos

El exit status se usa directamente con los operadores lógicos:

```
mkdir test && echo "Carpeta creada" || echo "No se pudo crear"
```

Comportamiento:

- `&&` → ejecuta el siguiente comando solo si el anterior tuvo éxito ($\$? = 0$)
- `||` → ejecuta el siguiente comando solo si el anterior falló ($\$? \neq 0$)

02.0 Que son las expansiones de la shell

Cuando escribimos un comando en Bash, **lo que vemos no siempre es exactamente lo que se ejecuta**.

Antes de enviar el comando al sistema, **la shell procesa la línea y expande ciertos símbolos y construcciones**.

Este proceso se llama **expansión**.

La shell **siempre** trata la línea primero y solo después ejecuta el comando resultante.

Definición

Una **expansión** es la sustitución de un símbolo o patrón por su **valor real**, antes de ejecutar el comando.

Ejemplos básicos de expansión

Expansión de tilde:

```
echo ~
```

Se expande a algo similar a:

```
/home/sec2john
```

Expansión de llaves:

```
echo {1..3}
```

Se expande a:

```
1 2 3
```

Expansión de variables:

```
echo $USER
```

Se expande al nombre del usuario actual.

Tipos de expansiones en Bash

Expansión de variables y parámetros

```
$USER  
${HOME}
```

Se sustituyen por el valor almacenado en la variable.

Expansión de llaves (brace expansion)

```
{a..z}
```

```
{lunes,martes}
```

Genera múltiples palabras a partir de un patrón.

Expansión de tilde

```
~
```

Se expande al directorio personal del usuario.

Expansión aritmética

```
$((1+2))
```

Se evalúa y se sustituye por el resultado:

```
3
```

Sustitución de comandos

```
$(date)
```

Se sustituye por la salida del comando ejecutado.

Globbing (expansión de archivos)

```
*.txt
```

Se expande a la lista de archivos que coinciden con el patrón.

Resumen

Las expansiones:

- Ocurren **antes** de ejecutar el comando
- Son responsabilidad de la **shell**
- Modifican la línea original
- Permiten escribir comandos más potentes y expresivos

Comprender las expansiones es fundamental para entender **qué ejecuta realmente Bash**.

02.1 Expansión de variables y parámetros en Bash

La expansión de variables y parámetros permite **manipular valores directamente desde la shell**, antes de ejecutar el comando.

Se realiza usando la sintaxis con `$` y, preferiblemente, llaves `{}`.

1. Expansión simple

```
$var
```

```
${var}
```

Ambas son equivalentes, pero:

- `${var}` es más **segura**
- Evita ambigüedades en expresiones complejas
- Es la forma recomendada en scripts

2. Valores por defecto y control de variables nulas

Estas expansiones permiten manejar variables **no definidas** o **vacías**.

```
${var:-word}
```

Usa `word` si `var` no existe o es nula.

```
${var:=word}
```

Asigna `word` a `var` si no existe o es nula.

```
${var:+word}
```

Usa `word` si `var` existe y no es nula.

```
${var:?message}
```

Muestra `message` y aborta si `var` no existe o es nula.

Ejemplo completo:

```
unset foo
echo ${foo:-"default"} #imprime "default"
echo ${foo:="assigned"} #imprime "assigned" y asigna el valor a foo
foo="bar"
echo ${foo:+alt} #imprime "alt"
echo ${baz:? "baz is missing"} #imprime "baz is missing"
```

3. Expansión de subcadenas (substring)

Permite extraer partes de una variable.

```
${var:offset}
```

```
${var:offset:length}
```

Ejemplo:

```
text="abcdef"
echo ${text:2} #imprime cdef
echo ${text:2:3} #imprime cde
echo ${text: -2} #imprime ef
```

Notas:

- El índice comienza en 0
- Índices negativos cuentan desde el final

4. Búsqueda y reemplazo

Permite sustituir patrones dentro del valor de una variable.

```
${var/pattern/replacement}
```

```
${var//pattern/replacement}
```

```
${var/#pattern/replacement}
```

```
${var/%pattern/replacement}
```

Ejemplo:

```
str="foo bar foo"
echo ${str/foo/baz} #imprime baz bar foo
echo ${str//foo/baz} # imprime baz bar baz
echo ${str/#foo/baz} # imprime baz bar foo
echo ${str/%foo/baz} # imprime foo bar baz
```

5. Eliminación de prefijo y sufijo

Permite eliminar partes del inicio o final de una variable.

```
${var#pattern}
```

```
${var##pattern}
```

```
${var%pattern}
```

```
${var%%pattern}
```

Ejemplo:

```
path="/home/user/docs/file.txt"
echo ${path#*/} # imprime 'home/user/docs/file.txt'
echo ${path##*/} # imprime 'file.txt'
echo ${path%/*} # imprime '/home/user/docs'
echo ${path%%/*} # imprime ''
```

6. Modificaciones de mayúsculas y minúsculas

Permite transformar el texto contenido en una variable.

```
${var^}
```

```
${var^^}
```

```
${var,}
```

```
${var,,}
```

Ejemplo:

```
str="hello world"
echo ${str^} # imprime 'Hello world'
echo ${str^^} # imprime 'HELLO WORLD'
echo ${str,} # imprime 'hello world'
echo ${str,,} # imprime 'hello world'
```

Idea clave

Las expansiones de parámetros:

- Ocurren antes de ejecutar el comando
- Evitan llamadas a comandos externos
- Son rápidas y potentes
- Son fundamentales para scripting avanzado en Bash

02.2 Brace expansion (expansión de llaves)

Brace expansion (expansión de llaves) en Bash

La **brace expansion** permite generar múltiples cadenas de texto a partir de un **patrón compacto**.

Se utiliza habitualmente para:

- Crear archivos o directorios
- Generar combinaciones
- Evitar escribir comandos repetitivos

La expansión se realiza **antes** de ejecutar el comando.

Sintaxis básica

Forma general:

```
{a,b,c}
```

Ejemplos:

```
echo {a,b,c} # a b c
```

```
echo folder_{a,b,c} #folder_a folder_b folder_c
```

```
echo {a,b,c}.txt #a.txt b.txt c.txt
```

Secuencias numéricas y alfabéticas

Forma general:

```
{inicio..fin}
```

Ejemplos numéricos:

```
echo {1..15} # 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

```
echo {1..0015} # 0001 0002 0003 0004 0005 0006 0007 0008 0009 0010 0011
0012 0013 0014 0015
```

El padding de ceros se conserva automáticamente.

Ejemplos alfabéticos:

```
echo {a..z} #a b c d e f g h i j k l m n o p q r s t u v w x y z
```

```
echo {M..Y} #M N O P Q R S T U V W X Y
```

Secuencias con incrementos

Forma general:

```
{inicio..fin..incremento}
```

Ejemplos:

```
echo {1..15..2} #1 3 5 7 9 11 13 15
```

```
echo {a..z..4} #a e i m q u y
```

Combinaciones de expansiones

Las expansiones pueden combinarse entre sí:

```
echo folder_{a,b,c}{1..5} # folder_a1 folder_a2 folder_a3 folder_a4
folder_a5 folder_b1 folder_b2 folder_b3 folder_b4 folder_b5 folder_c1
folder_c2 folder_c3 folder_c4 folder_c5
```

Esto genera todas las combinaciones posibles entre ambos conjuntos.

Idea clave

La brace expansion:

- No depende del sistema de archivos
- No usa variables
- Genera texto de forma puramente sintáctica

- Es una de las expansiones más potentes y rápidas de Bash

02.3 Expansión de tilde

En Bash, el símbolo `~` **no es decorativo**: es un **atajo a directorios**.

Antes de ejecutar un comando, la shell **expande automáticamente** la tilde al directorio correspondiente.

Esta expansión ocurre **antes** de que el comando se ejecute.

Tilde básica

La tilde sola representa el **directorio personal del usuario actual**.

```
~
```

Ejemplo:

```
echo ~
```

Salida típica:

```
/home/sec2john
```

Relación con la variable HOME

La tilde es equivalente a usar la variable de entorno `HOME`.

```
echo $HOME
```

Ambas expresiones apuntan al mismo directorio.

Acceder al home de otro usuario

También es posible usar la tilde con un nombre de usuario:

```
echo ~root
```

Esto se expande al directorio personal del usuario indicado (si existe y es accesible).

Uso práctico con rutas

La tilde se puede combinar con rutas relativas:

```
ls ~/Documentos
```

La shell expande primero `~` y luego evalúa el resto de la ruta.

02.4 Expansión aritmética

Bash no es una calculadora avanzada, pero permite realizar **operaciones aritméticas básicas** mediante la **expansión aritmética**.

Este tipo de expansión se evalúa **antes de ejecutar el comando**.

Expansión aritmética básica

La sintaxis general es:

```
$(( expresión ))
```

Ejemplo:

```
echo $((1+2)) #3
```

Uso con variables

La expansión aritmética permite mezclar valores literales y variables:

```
var=4  
echo $((4*$var)) #16
```

Dentro de la expresión:

- No es necesario usar `$` delante de las variables (aunque se permite)
- Se evalúa como una expresión matemática

Operaciones permitidas

Bash soporta:

- Suma, resta, multiplicación, división
- Módulo (o resto de división entera)
- Incrementos y decrementos
- Comparaciones
- Operadores lógicos y bit a bit

La lista completa está documentada en el manual oficial de Bash:

<https://www.gnu.org/software/bash/manual/bash.html#Shell-Arithmetic>

El comando integrado (())

Además de la expansión aritmética, Bash dispone del **comando integrado**:

```
(( expresión ))
```

Diferencias clave:

- (()) **evalúa en silencio**
- No produce salida
- Devuelve un estado de salida
- Es ideal para asignaciones y control de flujo (Explicado mas adelante)

Asignación y operaciones con (())

Ejemplos:

```
(( x=5 ))
```

```
(( x++ ))
```

```
(( y=x*2 ))
```

```
echo $x $y
```

Salida esperada:

```
6 12
```

Cuándo usar cada forma

Usa **expansión aritmética** cuando quieras **obtener un valor**:

```
${( ( ) )}
```

Usa **doble paréntesis** cuando quieras **evaluar, asignar o controlar flujo**:

```
(( ( ) ))
```

Idea clave

- `$(( ))` devuelve texto
- `(( ))` trabaja internamente
- Ambas forman parte del sistema aritmético de Bash
- Son fundamentales para scripting con lógica y cálculos

02.5 Sustitución de comandos \$(...)

La **sustitución de comandos** permite ejecutar un comando y **usar su salida como texto** dentro de otro comando o asignación.

Es como decirle a Bash:

| ejecuta esto primero y pega el resultado aquí

Es una de las características **más potentes y populares** del lenguaje Bash.

Sintaxis recomendada

La forma moderna y recomendada es:

```
$(comando)
```

Ejemplo:

```
echo "Estamos en el mes $(date +%m)"
```

La salida del comando `date` se sustituye directamente en la cadena.

Uso en scripts

Cualquier comando que produzca salida en la terminal puede:

- Capturarse en una variable
- Procesarse más adelante
- Integrarse en otro comando

Ejemplo:

```
resultado=$(ls)
```

Anidamiento de sustituciones

La sustitución de comandos **permite anidamiento**, lo que la hace muy flexible.

```
echo "Número de procesos: $(echo $(ps aux | wc -l))"
```

Cada sustitución se evalúa de dentro hacia fuera.

Ejecución en subshell

Cada sustitución de comandos:

- Se ejecuta en una **subshell** (Recuerda, una copia del entorno de ejecución de la shell padre)
- Hereda el entorno
- No puede modificar el entorno del proceso padre

Esto es importante cuando se trabaja con variables o cambios de entorno.

Sintaxis antigua con backticks

También existe la forma antigua:

```
`comando`
```

Ejemplo:

```
echo "Hoy es `date`"
```

Problemas de esta sintaxis:

- Difícil de leer
- Difícil de mantener
- No permite anidamiento limpio

Recomendación

Usa **siempre**:

```
$(comando)
```

Evita los backticks salvo en casos muy concretos.

Idea clave

La sustitución de comandos:

- Convierte la salida de un comando en texto reutilizable
- Permite composición avanzada de comandos

- Es fundamental para scripting dinámico en Bash

02.6 Word Splitting (división en palabras)

El **word splitting** es una de las expansiones más **transparentes y problemáticas** en Bash.

Bash **necesita separar el texto en “palabras”** para poder ejecutar comandos correctamente.

Cuando escribes una línea, Bash no ve una cadena única, sino **una lista de palabras (tokens)**.

Separadores por defecto

Por defecto, Bash usa como separadores:

- Espacios
- Tabulaciones
- Saltos de línea

Estos separadores están definidos en la variable especial:

```
IFS
```

Para ver su contenido:

```
echo -n "$IFS" | od -c
```

Sin word splitting, Bash no podría separar:

- El comando
- Sus argumentos

Cuándo ocurre el word splitting

Después de:

- Expansión de variables
- Expansión aritmética
- Sustitución de comandos

Bash realiza un paso **oculto** llamado **word splitting**, donde divide el texto resultante usando `IFS`.

Esto puede **romper scripts** si no se controla correctamente.

Ejemplo básico

```
VAR="mi variable larga"
```

Sin comillas:

```
echo $VAR
```

Bash interpreta:

- 3 palabras
- 3 argumentos para `echo`

Con comillas:

```
echo "$VAR"
```

Bash interpreta:

- 1 sola palabra
- 1 argumento

Problema real en scripts

Con `echo` normalmente no hay problema, pero en operaciones reales sí.

Ejemplo con nombres de archivos:

```
VAR="mi fichero raro.txt"
```

Redirección correcta:

```
cat > "/tmp/$VAR.txt"
```

Uso incorrecto (rompe por word splitting):

```
cat $VAR
```

Uso correcto:

```
cat "$VAR"
```

Recomendación clave

Siempre que no quieras que Bash divida el contenido de una variable:

- **Protégela con comillas dobles**

Regla práctica:

Si usas una variable, **cítala**, salvo que tengas un motivo claro para no hacerlo.

Idea clave

El word splitting:

- Es automático
- Depende de `IFS`
- Ocurre tras otras expansiones
- Es una fuente común de errores en scripts
- Se controla usando comillas correctamente

02.7 Globbing (expansión de nombres de fichero)

El **globbing** es una de las expansiones más usadas en la línea de comandos.

Permite que Bash **reemplace patrones** por los **nombres de archivos o directorios que coinciden**, antes de ejecutar el comando.

Es, en la práctica, un sistema de **comodines** interpretado por la shell.

Caracteres básicos de globbing

```
*
```

Coincide con **cualquier número de caracteres**, incluido ninguno (la cadena vacía)

```
?
```

Coincide con **un solo carácter**.

```
[ ]
```

Coincide con **un carácter cualquiera dentro del conjunto** indicado.

Ejemplos con *

```
ls *.txt
```

```
cd /usr/bin  
ls apt*  
ls xdg-*  
ls a*d
```

En todos los casos, Bash sustituye el patrón por la lista de archivos coincidentes.

Ejemplos con ?

```
cd /usr/share/bash-completion/completions/  
ls -l python?  
ls -l python3.?
```

El carácter `?` exige **exactamente un carácter** en esa posición.

Ejemplos con `[ ]`

```
cd /usr/share/bash-completion/completions/  
file [_7]*  
file [_7][isxy]*
```

Los corchetes permiten:

- Definir conjuntos de caracteres
- Crear patrones más precisos

Uso en scripts

El globbing funciona:

- En la shell interactiva
- Dentro de scripts

Siempre que Bash esté procesando la línea de comandos.

Advertencia importante

Aunque se parezca visualmente:

- **El globbing NO es una expresión regular**
- Se procesa por la shell, no por el comando
- Tiene reglas y comportamiento distintos a las regex

Confundir globbing con expresiones regulares es una fuente común de errores.

Idea clave

El globbing:

- Es una expansión previa a la ejecución
- Trabaja sobre el sistema de archivos
- Usa patrones simples y potentes
- Es fundamental para trabajar con conjuntos de archivos

02.8 Dispositivos especiales

En Linux, **casi todo se maneja como si fuera un fichero**: tanto los archivos normales como los dispositivos de hardware o los canales de comunicación entre procesos.

Dentro del directorio `/dev` existen muchos de estos *ficheros especiales*. Entre ellos, tres que están siempre disponibles para cualquier programa:

- **stdin** (entrada estándar)
- **stdout** (salida estándar)
- **stderr** (salida de errores)

Si miramos en `/dev` veremos algo así:

```
$ ls -l /dev/std*  
lrwxrwxrwx 1 15 /dev/stdout -> /proc/self/fd/1  
lrwxrwxrwx 1 15 /dev/stdin -> /proc/self/fd/0  
lrwxrwxrwx 1 15 /dev/stderr -> /proc/self/fd/2
```

Estos ficheros son en realidad **enlaces** a `/proc/self/fd/`, un directorio especial que muestra los “ficheros abiertos” por el proceso actual.

- Un descriptor de fichero es simplemente un **número** que el sistema operativo usa para referirse a un recurso abierto.
- Los tres primeros (0, 1 y 2) siempre están reservados:
 - 0 → stdin (entrada)
 - 1 → stdout (salida normal)
 - 2 → stderr (salida de errores)

En un terminal:

- **stdin** está conectado al teclado (lo que escribes).
- **stdout** y **stderr** están conectados a la pantalla (lo que ves).

02.9 Shell Options (shopt)

En Bash existe un **comando integrado** llamado `shopt` (*shell options*) que permite **configurar el comportamiento de la shell**.

Estas opciones afectan a cómo Bash interpreta patrones, globbing y otros detalles internos, y son especialmente útiles en **scripts**.

Ver opciones disponibles

Si ejecutamos `shopt` sin argumentos, se muestran todas las opciones disponibles (más de 50):

```
shopt
```

Cada opción puede estar:

- Activada
- Desactivada

Activar y desactivar opciones

Activar una opción:

```
shopt -s nombre_opcion
```

Desactivar una opción:

```
shopt -u nombre_opcion
```

Las opciones pueden activarse:

- En una shell interactiva
- Dentro de un script (solo afectan al script)

Documentación oficial

El significado exacto de cada opción está documentado en el manual oficial de Bash:

<https://www.gnu.org/software/bash/manual/bash.html#The-Shopt-Builtin>

Opciones de shopt más usadas en scripts

dotglob

Si está activada, los ficheros ocultos (los que empiezan por `.`) se incluyen en el globbing.

```
shopt -s dotglob
```

Sin `dotglob`, los archivos ocultos **no** coinciden con `*`.

nullglob

Si está activada, los patrones de globbing que **no coinciden con ningún fichero** se expanden a **nada**.

Comportamiento por defecto:

```
echo *.txt
```

Resultado literal si no hay coincidencias:

```
*.txt
```

Con `nullglob` activado:

```
shopt -s nullglob  
echo *.txt
```

Resultado:

- No produce salida
- El patrón se elimina

Es muy útil para evitar procesar patrones inexistentes.

failglob

Si está activada, un patrón que no coincide con ningún fichero provoca un **error en la expansión**.

```
shopt -s failglob
```

Comportamiento:

- Bash genera un error
- El script puede abortar

Útil cuando:

- Esperas que existan ciertos ficheros
- Quieres que el script falle si no están presentes

globstar

Permite usar `**` para recorrer directorios **de forma recursiva**.

```
shopt -s globstar
```

Ejemplos conceptuales:

- `**` → archivos y directorios recursivamente
- `**/` → solo directorios

Ideal para:

- Procesar árboles de directorios
- Evitar el uso de `find` en scripts sencillos

nocaseglob

Hace que el globbing sea **insensible a mayúsculas y minúsculas**.

```
shopt -s nocaseglob
```

Ejemplo:

- `*.jpg` coincide con `foto.JPG`, `foto.jpg`, `F0T0.Jpg`

Muy útil en scripts que procesan ficheros sin controlar el case.

Idea clave

`shopt` permite:

- Ajustar el comportamiento de Bash
- Hacer scripts más robustos
- Controlar cómo se expanden los patrones

Activar la opción adecuada puede:

- Evitar errores silenciosos
- Forzar fallos controlados

- Simplificar lógica en scripts

03.0 Que es el control de flujo

Hasta ahora, los scripts se ejecutan **de arriba a abajo**, instrucción tras instrucción. Este comportamiento se llama **flujo secuencial**.

Pero en scripting necesitamos algo más que ejecutar líneas en orden.

¿Qué pasa si queremos:

- Ejecutar un comando **solo si** se cumple una condición
- Repetir una acción **hasta** que ocurra algo
- Salir del script antes de terminar

Para eso existe el **control de flujo**.

¿Qué es el control de flujo?

El **control de flujo** permite decidir:

- Qué instrucciones se ejecutan
- Cuántas veces se ejecutan
- Cuándo se interrumpe la ejecución

Un script no solo ejecuta comandos: **toma decisiones**.

Conceptos básicos del control de flujo

Condicionales

Permiten ejecutar comandos **solo si** una condición se cumple.

Ejemplos conceptuales:

- Si un archivo existe
- Si un comando falla
- Si un valor es mayor que otro

Bucles

Permiten **repetir comandos** mientras una condición sea verdadera o durante un conjunto de valores.

Ejemplos de uso:

- Procesar líneas de un fichero

- Recorrer parámetros
- Repetir tareas automatizadas

Salidas y control de ejecución

Permiten **alterar o interrumpir** el flujo normal del script.

Palabras clave habituales:

```
break
continue
exit
```

Se usan para:

- Salir de un bucle
- Saltar a la siguiente iteración
- Finalizar el script inmediatamente

Relación con el exit status

Internamente, Bash toma decisiones basándose en el **estado de salida** de los comandos.

Recordatorio:

- 0 → éxito
- distinto de 0 → error

Ese valor es la base para:

- Condicionales
- Bucles
- Encadenamiento lógico de comandos

El control de flujo en Bash se apoya directamente en el **exit status** para decidir qué camino seguir.

Idea clave

El control de flujo permite que un script:

- No sea solo secuencial
- Tome decisiones
- Reaccione a errores
- Repita tareas de forma controlada

Es el paso que convierte un conjunto de comandos en un **programa real**.

03.1 Condicionales

Un **condicional** (o sentencia condicional) es un constructo que permite **tomar decisiones** en función de si una expresión se evalúa como **verdadera** o **falsa**.

En Bash, las condiciones se evalúan usando el **estado de salida**:

- 0 → condición verdadera
- distinto de 0 → condición falsa

Formas de evaluar condiciones en Bash

Existen **tres formas principales** de evaluar expresiones condicionales:

- Comando integrado `test`
- Constructo `[ ... ]`
- Keyword `[[ ... ]]`

Todas reciben una expresión y devuelven un estado de salida.

Ejemplo básico con strings

Vamos a comprobar si una variable contiene un valor concreto.

```
var="abcde"
test $var = "abcde"
echo $?
```

Resultado:

- 0 → la condición es verdadera

Caso con variable vacía:

```
var=
test $var = "abcde"
echo $?
```

Resultado:

- valor distinto de 0 → condición falsa

Uso de `[ ... ]`

El constructo `[ ... ]` es **equivalente a** `test`.

```
[ $var = "abcde" ] && echo $?
```

Uso de `[[ ... ]]`

```
[[ $var = "abcde" ]] && echo $?
```

Características de `[[ ... ]]`:

- Es una **keyword de la shell**, no un comando
- Tiene más capacidades que `[ ... ]`
- Mejor manejo de strings y variables
- Mejor rendimiento
- Menos portable entre sistemas

Se estudiará en profundidad en condicionales avanzados.

Relación entre `test` y corchetes

- `test` y `[ ... ]` son **totalmente equivalentes**
- `[[ ... ]]` es más potente y segura, pero específica de Bash

Tipos de expresiones condicionales

Bash permite evaluar distintos tipos de expresiones:

- **Ficheros**
- **Strings**
- **Números**

Forma general para comparaciones numéricas:

```
arg1 OP arg2
```

La lista completa de expresiones condicionales está documentada en el manual oficial de Bash:

<https://www.gnu.org/software/bash/manual/bash.html#Bash-Conditional-Expressions>

03.2 Condicionales simples. if, then, else

La idea básica de un condicional es:

“Si ocurre algo → haz esto.

Si no → haz otra cosa.”

Ya conocemos cómo **evaluar expresiones condicionales**.

Ahora vemos cómo **controlar el flujo** usando la estructura `if`.

Forma básica del if

La forma más sencilla del condicional es:

```
if test-commands;  
then  
    commands  
fi
```

`test-commands` puede ser cualquiera de:

- `test`
- `[ ... ]`
- `[[ ... ]]`

Ejemplo básico

Ejecutado directamente en la shell:

```
if test -f /etc/passwd;  
then  
    echo "it's a file"  
fi
```

Si la condición es verdadera (exit status `0`), se ejecuta el bloque `then`.

Variante if-else

Permite ejecutar un bloque alternativo si la condición no se cumple.

```
if test-commands;  
then  
    commands
```

```

else
    commands
fi

```

Ejemplo:

```

if test -d /etc/passwd;
then
    echo "it's a directory"
else
    echo "it is not a directory"
fi

```

Variante if-elif-else (escalera if-then-else)

Permite evaluar **varias condiciones en orden**.

```

if test-commands;
then
    commands
elif test-commands;
then
    commands
elif test-commands;
then
    commands
else
    commands
fi

```

Ejemplo:

```

if test -f /dev/sda;
then
    echo "it's a file"
elif [ -d /dev/sda ];
then
    echo "it's a directory"
elif [[ -c /dev/sda ]];
then
    echo "it's a character special file"
elif [ -b /dev/sda ];

```

```
then
    echo "it's a block special file"
fi
```

Las condiciones se evalúan **en orden**, y solo se ejecuta el primer bloque que resulte verdadero.

Definición formal según el manual de Bash

La estructura general del `if` es:

```
if test-commands; then
    consequent-commands;
[elif more-test-commands; then
    more-consequents;]
[else alternate-consequents;]
fi
```

Las partes entre corchetes son **opcionales**.

03.3 Condicionales avanzados

En esta lección se explican los detalles de la **palabra clave** `[[ ... ]]` y cómo construir **condicionales avanzados** combinando expresiones lógicas como AND, OR y NOT.

Shell keyword `[[ ... ]]`

El operador `[ ... ]` es histórico y proviene de la Bourne shell (`sh`).

El operador `[[ ... ]]` es posterior (introducido en 1988 por KornShell) y fue adoptado por Bash.

`[[ ... ]]` **no es un comando**, es una **keyword de la shell**, lo que le da más capacidades y seguridad.

Diferencias entre `[ ... ]` y `[[ ... ]]`

Origen:

- `[ ... ]` → comando `test` (POSIX)
- `[[ ... ]]` → sintaxis interna de Bash (no POSIX)

Compatibilidad:

- `[ ... ]` funciona en cualquier shell POSIX
- `[[ ... ]]` solo en Bash, KornShell y Zsh

Manejo de variables

Con `[ ... ]` es obligatorio citar (Entrecomillar doble) variables:

```
var="Mi var"
[ "$var" = "Mi var" ] && echo "todo bien"
```

Con `[[ ... ]]` no es obligatorio:

```
var="Mi var"
[[ $var == "Mi var" ]] && echo "todo bien"
```

Operadores lógicos

Con `[ ... ]` se usan operadores antiguos:

```
[ -r /etc/passwd -a -f /etc/passwd ] && echo "OK"
```

Con `[[ ... ]]` se usan operadores modernos y claros:

```
[[ -r /etc/passwd && -f /etc/passwd ]] && echo "OK"
```

Comparaciones de cadenas

Ambos soportan igualdad:

```
[ "$var" = "Mi var" ]
```

```
[[ $var == "Mi var" ]]
```

Pero `[[ ... ]]` además soporta **patrones (globbing)**:

```
[[ $var == "Mi "* ]] && echo "Match!"
```

Expresiones regulares

Solo `[[ ... ]]` soporta regex con `==~`:

```
var="12345678"
[[ $var ==~ 2345 ]] && echo "Match!"
```

Seguridad y robustez

`[[ ... ]]:`

- No necesita tantas comillas
- No se rompe con espacios o variables vacías
- No expande globs a archivos reales
- Permite expresiones más complejas

Siempre que no necesites **compatibilidad POSIX**, es la opción recomendada.

Condicionales compuestos

Un condicional no tiene por qué limitarse a una sola expresión.
Podemos **combinar varias condiciones** en una sola evaluación.

Ejemplo conceptual:

- Si el fichero es regular
- Y se puede escribir

- Y no es fin de mes

Operadores lógicos

NOT

```
! EXPRESSION
```

Niega el resultado de la expresión.

AND lógico

```
EXPRESSION1 && EXPRESSION2
```

La condición es verdadera solo si **ambas** expresiones son verdaderas.

OR lógico

```
EXPRESSION1 || EXPRESSION2
```

La condición es verdadera si **alguna** de las expresiones es verdadera.

Ejemplo de condicional compuesto

```
[[ -f archivo.txt && -w archivo.txt ]] && echo "Archivo válido"
```

Idea clave

- `[[ ... ]]` es más potente y seguro que `[ ... ]`
- Permite patrones, regex y operadores modernos
- Los condicionales compuestos permiten expresar lógica compleja
- Es la base para scripts robustos y mantenibles en Bash

03.4 Case

El constructo `case` permite manejar **múltiples condiciones** de forma clara y estructurada, evitando largas escaleras de `if / elif / else`.

Es especialmente útil cuando:

- Una variable o parámetro puede tomar **muchos valores posibles**
- Queremos comparar contra **patrones**, no solo igualdad exacta

Sintaxis básica

```
case word in
  patron1) comandos...;;
  patron2) comandos...;;
  *) comandos_por_defecto...;;
esac
```

Elementos del case

1. `case` y `esac`

Son **palabras clave** (keywords) del intérprete de Bash.

2. `word`

Es el valor que se va a evaluar (normalmente una variable o parámetro).

Antes de comparar, Bash **expande** en `word`:

- Tilde
- Variables y parámetros
- Sustitución de comandos
- Expansión aritmética

3. **Patrones**

Cada cláusula contiene uno o varios **patrones** que intentan coincidir con `word`.

Los patrones usan **globbing**:

```
*
?
[...]
```

También se expanden antes de evaluarse.

4. `;;`

Marca el final de una cláusula.

Comportamiento del matching

- Bash evalúa los patrones **en orden**
- **El primer patrón que coincide se ejecuta**
- El resto de cláusulas se ignoran (salvo usos avanzados)

Ejemplo simple

```
case "$1" in
  start) echo "Arrancando";;
  stop)  echo "Deteniendo";;
  *)     echo "Opción desconocida";;
esac
```

Sintaxis avanzada

```
case word in
  patron1 | patron2 | patron3) comandos...;;
  patron4) comandos...;&
  patron5) comandos...;&
  *) comandos_por_defecto...;;
esac
```

Patrones múltiples (OR)

Se pueden combinar patrones usando `|` :

```
case "$1" in
  start | run | begin) echo "Iniciando";;
esac
```

Esto equivale a un OR lógico entre patrones.

Terminadores avanzados

Además de `;;` , existen otros terminadores:

`;;&`

`;;&`

- Tras ejecutar la cláusula actual

- Bash **evalúa el patrón de la siguiente cláusula**
- Si hay match, ejecuta también esa cláusula

```
; &
```

```
; &
```

- Tras ejecutar la cláusula actual
- Bash **ejecuta directamente** la siguiente cláusula
- No se evalúa el patrón siguiente

Cuándo usar case

case es ideal cuando:

- Comparas una variable contra muchos valores
- Trabajas con opciones o comandos (start , stop , restart)
- Necesitas patrones en lugar de igualdad exacta
- Quieres código más legible que múltiples if

03.5 for

Bucle for en Bash

Para repetir un bloque de instrucciones un **número finito de veces**, Bash ofrece el **bucle for**.

Se usa cuando:

- Sabemos **dónde empezamos y dónde acabamos**
- Queremos iterar sobre una **lista de valores**

Sintaxis básica del for

```
for variable in lista; do  
    comandos  
done
```

Elementos del bucle

1. **for / do / done**

Son palabras clave de Bash que delimitan el bucle.

2. **variable**

Toma el valor de cada elemento de la lista en cada iteración.

3. **lista**

Es el conjunto de valores a recorrer.

La lista:

- Se expande usando **todas las expansiones de Bash**
- Puede generarse dinámicamente
- Puede provenir de globbing, brace expansion, sustitución de comandos, etc.

Ejemplo simple

```
for i in a b c; do  
    echo "$i"  
done
```

Lista generada dinámicamente

La lista puede venir de una expansión o de un comando:

```
for f in *.txt; do #toma los ficheros *.txt del directorio en el que se
ejecuta
    echo "$f"
done
```

```
for u in $(cut -d: -f1 /etc/passwd); do #toma el primer campo del fichero
/etc/passwd, es decir, el nombre del usuario.
    echo "$u"
done
```

Sintaxis estilo C (for aritmético)

Bash también ofrece una forma similar a lenguajes como C o Java:

```
for (( expr1 ; expr2 ; expr3 )); do
    commands
done
```

Donde:

- `expr1` → inicialización
- `expr2` → condición
- `expr3` → incremento/decremento

Ejemplo con for aritmético

```
for (( i=0; i<5; i++ )); do
    echo "$i"
done
#desde i igual a cero, mientras i sea menor que 5, i incrementa
```

Diferencias entre ambos for

- `for variable in lista`
 - Ideal para recorrer listas, archivos, parámetros
- `for (( ... ))`
 - Ideal para contadores y rangos numéricos

- Usa aritmética interna de Bash

03.6 while y until

Los bucles `while` y `until` permiten repetir acciones **mientras** o **hasta que** se cumpla una condición.

Son especialmente útiles en scripts que:

- Leen información
- Monitorizan estados
- Esperan a que ocurra un evento

Bucle while

El bucle `while` se ejecuta **mientras la condición sea verdadera**.

```
while comando; do
    comandos
done
```

- El comando se evalúa antes de cada iteración
- Si devuelve exit status `0`, el bucle continúa
- Si devuelve distinto de `0`, el bucle termina

Bucle until

El bucle `until` es el inverso de `while`.

```
until comando; do
    comandos
done
```

- El bucle se ejecuta **mientras la condición sea falsa**
- Termina cuando el comando devuelve exit status `0`

Exit status de while y until

El estado de salida de un bucle `while` o `until` es:

- El **exit status del último comando ejecutado dentro del bucle**

Esto es relevante cuando:

- Encadenamos comandos

- Evaluamos el resultado del bucle

Bucles infinitos

Un bucle es **infinito** si la condición que lo controla **nunca cambia**.

Ejemplo típico:

```
while true; do
  comandos
done
```

Este bucle solo terminará si:

- Se ejecuta `break` dentro del mismo en algún punto.
- El script es interrumpido

Advertencia

Los bucles infinitos:

- Pueden bloquear scripts
- Consumir recursos innecesariamente

Siempre deben incluir:

- Una condición de salida
- O una interrupción explícita

Resumen

- `while` repite mientras la condición sea verdadera
- `until` repite hasta que la condición sea verdadera
- Ambos dependen del exit status
- Son fundamentales para scripting reactivo y dinámico

03.7 Select

El constructo `select` permite crear **menús interactivos** de forma muy rápida.

Muestra al usuario una lista de opciones y espera a que seleccione una.

Internamente, `select` es un **bucle** que se repite hasta que se interrumpe.

Sintaxis básica

```
select variable in lista; do
    comandos
done
```

Elementos del select

- `select / do / done`
Palabras clave de Bash que delimitan el bucle.
- `variable`
Recibe el valor de la opción seleccionada.
- `lista`
Conjunto de opciones que se mostrarán como menú.

Bash muestra automáticamente:

- Un número para cada opción
- Un prompt solicitando la selección

Variable especial PS3

Bash utiliza la variable `PS3` para mostrar el texto del prompt del menú.

Puede definirse antes o durante la ejecución del script.

```
export PS3="Please choose an option: "
```

Ejemplo de ejecución:

```
./3.7_script.sh
```

Si no se define `PS3`, Bash usa un valor por defecto.

Ejemplo simple

```
select opt in start stop restart; do
    echo "Has elegido: $opt"
done
```

El bucle se repite hasta que:

- Se interrumpa (ctrl + c)

Combinación potente: select + case

Una de las combinaciones más habituales y potentes es `select` junto con `case`, ideal para menús reales.

Estructura típica:

```
select opt in start stop restart exit; do
    case "$opt" in
        start)    echo "Arrancando";;
        stop)     echo "Deteniendo";;
        restart)  echo "Reiniciando";;
        exit)     break;;
        *)        echo "Opción inválida";;
    esac
done
```

Esto permite:

- Menús claros
- Lógica estructurada
- Código legible y mantenible

Listas dinámicas en select

La lista de opciones puede generarse dinámicamente, por ejemplo a partir de archivos:

```
select f in *; do
    echo "Seleccionado: $f"
done
```

Advertencia importante

Usar `*` para seleccionar archivos **no es la forma más segura**:

- Los nombres con espacios pueden causar problemas

- El globbing puede romper selecciones complejas

Este enfoque debe usarse con cuidado y conocimiento del contexto.

Idea clave

El constructo `select` :

- Crea menús interactivos rápidamente
- Es un bucle por naturaleza
- Se combina muy bien con `case`
- Es ideal para scripts interactivos
- Requiere cuidado al trabajar con nombres de archivos

03.8 break, continue y return

Bash dispone de **tres comandos integrados** que permiten controlar el flujo de ejecución **dentro de bucles y funciones**:

- `break`
- `continue`
- `return`

Estos comandos no ejecutan acciones externas, sino que **modifican el flujo del script**.

break

`break` permite **salir inmediatamente del bucle actual**.

- Afecta al **bucle más interno**
- Es útil cuando ya no tiene sentido seguir iterando

Ejemplo conceptual:

- Buscar un valor en una lista
- Cuando se encuentra, se abandona el bucle

Ejemplo:

```
for i in a b c d; do
    if [[ $i == "c" ]]; then
        break
    fi
    echo "$i"
done
```

En este caso, el bucle termina al llegar a `c`.

continue

`continue` permite **saltar a la siguiente iteración** del bucle.

- El bucle no termina
- Se ignora el resto de instrucciones de la iteración actual

Se usa cuando queremos **descartar el valor actual**, pero seguir iterando.

Ejemplo:

```
for i in a b c d; do
    if [[ $i == "b" ]]; then
        continue
    fi
    echo "$i"
done
```

Aquí, `b` se omite, pero el bucle continúa.

return

`return` devuelve un **valor de salida desde una función**.

```
return valor
```

Importante:

- `return` **solo debe usarse en funciones**
- Un script Bash **no puede devolver un valor con `return`** si se ejecuta como proceso independiente

return y scripts

Si un script se ejecuta como **proceso nuevo**:

```
bash script.sh
```

```
./script.sh
```

Entonces:

- `return` **no es válido**
- Provoca un error (visible en terminal)

return con source

Si el script se carga con:

```
source script.sh
```

o:

```
. script.sh
```

Entonces:

- El script se ejecuta en la **shell actual**
- `return` **sí funciona**
- Puede devolver control y valor a la shell llamante

exit en scripts

En scripts, para terminar la ejecución y devolver un estado al sistema, se usa:

```
exit valor
```

El valor devuelto puede consultarse con:

```
echo $?
```

Esto es equivalente a devolver un **exit status** al proceso padre.

Resumen rápido

- `break` → sale del bucle actual
- `continue` → pasa a la siguiente iteración
- `return` → devuelve valor desde una función
- `exit` → finaliza el script y devuelve exit status

03.9 Agrupaciones y listas de comandos

En Bash podemos **agrupar comandos** y **encadenarlos** para controlar cómo se ejecutan y cómo se propaga su estado de salida.

Agrupaciones de comandos

Existen **dos formas** de agrupar comandos:

- Con **paréntesis** (...)
- Con **llaves** { ...; }

La diferencia clave es **el entorno de ejecución**.

Agrupación con paréntesis (...)

Los paréntesis ejecutan los comandos en una **subshell** (proceso hijo).

Características:

- Se crea un proceso nuevo
- Los cambios **no afectan** al entorno padre
- Variables y cambios de directorio se pierden al terminar

Ejemplo conceptual (cambio de directorio no persistente):

```
( cd /tmp; pwd )  
pwd
```

El segundo `pwd` no refleja el cambio.

Agrupación con llaves { ...; }

Las llaves ejecutan los comandos en el **mismo proceso** (misma shell).

Características:

- No se crea subshell
- Los cambios **sí afectan** al entorno actual
- Comparten variables y estado

 **Detalles sintácticos importantes:**

- Debe haber espacios: { y }

- Debe terminar en `;` o salto de línea antes de `}`

Ejemplo:

```
{ cd /tmp; pwd; }
pwd
```

Aquí el cambio de directorio **sí persiste** porque afecta a `PWD`.

Cuándo usar agrupaciones

Usa agrupaciones cuando quieras:

- Agrupar lógicamente varios comandos
- Compartir variables entre comandos
- Controlar el contexto (subshell vs shell actual)

Listas de comandos

Las **listas de comandos** se construyen encadenando comandos con operadores.

Bash dispone de **cuatro operadores** principales:

- `;`
- `&`
- `&&`
- `||`

Operador `;`

Encadena comandos **sin importar el resultado** del anterior.

```
command1; command2
```

- `command2` se ejecuta siempre
- No hay evaluación lógica

Operador `&`

Ejecuta el comando en **background**.

```
command &
```


El comando:

- Se ejecuta en segundo plano
- No bloquea la shell

(Tema avanzado, se verá más adelante)

Operador `&&`

AND lógico basado en el **exit status**.

```
command1 && command2
```

Comportamiento:

- Si `command1` devuelve `0`
- Entonces se ejecuta `command2`

Operador `||`

OR lógico basado en el **exit status**.

```
command1 || command2
```

Comportamiento:

- Si `command1` devuelve distinto de `0`
- Entonces se ejecuta `command2`

Combinación de agrupaciones y listas

Podemos combinar agrupaciones con listas para tratar un bloque como una unidad lógica.

Ejemplo conceptual:

```
{ command1; command2; } && echo "Todo OK" || echo "Error"
```

Aquí:

- El bloque se evalúa como conjunto
- El operador lógico decide qué ejecutar después

Nota importante sobre el estado de salida

En una lista o agrupación:

- El estado de salida es el del **último comando ejecutado**

Si necesitamos:

- Detectar errores intermedios
- Considerar fallo si **cualquier** comando falla

Entonces debemos usar una **función**, que permite devolver un estado controlado.

(Se verá en la lección de funciones, módulo 4)

Resumen

- `( ... )` → subshell, no modifica el entorno padre
- `{ ...; }` → mismo entorno, los cambios persisten
- `;` encadena sin lógica
- `&&` y `||` encadenan según exit status
- Agrupaciones + listas permiten control fino del flujo

04.0 Introducción

En este módulo vamos a entrar en aspectos avanzados del scripting en bash, en concreto vamos a ver los siguientes:

Estructuras lógicas internas del lenguaje (funciones, arrays).

- Organización y estructuración del script mediante bloques de código o funciones
- Estructuras de datos lógicas: Arrays o arreglos. Indexados y asociativos.
- Variable de entorno IFS (Internal Field Separator)

Entrada/salida avanzada y manejo de descriptores.

- Redirección avanzada
- Comunicación entre procesos. Tuberías sin nombre (FIFOs)
- Creación y manejo de descriptores de fichero

Comunicación entre procesos, concurrencia y señales.

- Ejecución paralela de procesos, sustitución de procesos
- Control de procesos (control de jobs). Ejecución en background o segundo plano.
- Captura de señales. Ventajas en scripts.
- Comunicación bidireccional mediante coprocesos

04.1 Funciones

Aprender a **definir, invocar y gestionar funciones** en scripts Bash para:

- Modularizar el código
- Reutilizar tareas
- Mejorar la legibilidad y el mantenimiento

En Bash, una **función** es un bloque de comandos que se ejecuta **dentro del mismo entorno del script**, a diferencia de:

- Un subshell
- Un script externo

Definición de funciones

Existen dos formas válidas de definir una función.

Forma recomendada:

```
nombre_funcion() {  
    comandos  
}
```

Forma alternativa:

```
function nombre_funcion {  
    comandos  
}
```

 Detalle importante:

- Deben existir **espacios** entre `{` y `}`

Ejemplo básico en la shell

```
function hello { echo 'hello world!!!'; }
```

Invocación:

```
hello  
#salida: 'hello world!!'
```

Funciones con parámetros

Las funciones reciben parámetros posicionales igual que los scripts.

```
hello_advanced() { echo "hello $1"!!!; }
```

Invocación:

```
hello_advanced Juan
```

Parámetros disponibles:

- `$1` , `$2` , ... → parámetros individuales
 - `$@` o `"$@"` → todos los parámetros
-

Dónde se almacenan las funciones

Las funciones definidas existen en el entorno de la shell actual.

Para verlas:

```
set | grep -C 5 hello
```

```
declare -f | grep -C 5 hello
```

Eliminar funciones

Para eliminar una función definida:

```
unset -f hello
```

Esto elimina la función del entorno actual.

Alcance de variables (scope)

Por defecto:

- Las variables definidas dentro de una función son **globales**
- Comparten el mismo entorno que el script

Ejemplo conceptual:

- Una variable definida en una función puede sobrescribir otra global
-

Variables locales

Para evitar efectos colaterales, se usa el builtin `local`.

```
mi_funcion() {  
    local var="solo visible aquí"  
}
```

Comportamiento:

- La variable solo existe dentro de la función
 - Se destruye al finalizar la función
-

Retorno de funciones

Las funciones devuelven un **estado de salida**, igual que cualquier comando.

```
return valor
```

Características:

- El valor debe estar entre `0` y `255`
- `0` indica éxito
- Valores distintos de `0` indican error

 `return` **NO devuelve texto**, solo un número.

Devolver texto desde una función

Para devolver texto:

- Se imprime por stdout: `echo` o `printf`
- Se captura la salida

Ejemplo conceptual:

```
resultado=$(mi_funcion)
```

Funciones recursivas

Bash permite **funciones recursivas**, es decir, funciones que se llaman a sí mismas.

Ejemplo conceptual:

- Cálculo
- Procesamiento jerárquico
- Recorrido de estructuras

Idea clave

- Las funciones se ejecutan en el mismo entorno
- Aceptan parámetros posicionales
- Devuelven estados de salida
- `local` evita colisiones de variables
- Permiten código modular, reutilizable y claro

04.2 Arrays Indexados

Un **array indexado** es una variable que puede almacenar **varios valores**, cada uno asociado a un **índice numérico** que comienza en `0`.

Son útiles para:

- Manejar listas de datos
- Iterar sobre conjuntos de valores
- Evitar múltiples variables relacionadas

Declaración y asignación

Array vacío usando `declare`:

```
declare -a colores
```

Declaración directa con valores iniciales:

```
colores=("verde" "rosa" "rojo" "negro")
```

Asignación de elementos individuales:

```
colores[0]="Amarillo"  
colores[4]="Azul"
```

Declaración implícita sin inicializar previamente:

```
numeros[1]=10
```

Acceso a elementos y propiedades

Acceder a un elemento concreto:

```
echo "Elemento 1: ${colores[0]}"
```


Número total de elementos:

```
echo "Número de elementos: ${#colores[@]}"
```

Todos los elementos del array:

```
echo "Todos los elementos: ${colores[@]}"
```

Índices del array

Para obtener la lista de índices usados:

```
echo "${!colores[@]}"
```

Ejemplo con índices no consecutivos:

```
colores[7]="Naranja"  
echo "${!colores[@]}"
```

Nota:

- Los arrays en Bash **no necesitan índices consecutivos**
 - Un array puede tener “huecos”
-

Iterar sobre un array

Iterar sobre los valores:

```
for color in "${colores[@]"; do  
    echo "Color $color"  
done
```

Iterar sobre índices y valores:

```
for i in "${!colores[@]"; do  
    echo "Indice: $i - Color ${colores[$i]}"  
done
```

Añadir, modificar y eliminar elementos

Añadir elementos al final del array:

```
colores+=(violeta marrón)
```

Eliminar un elemento concreto:

```
unset 'colores[2]'
```

Modificar un elemento existente:

```
colores[3]="Gris"
```

Cargar arrays con readarray / mapfile

Desde un fichero

Leer líneas de un fichero en un array

(-t elimina el salto de línea final):

```
readarray -t lineas < /etc/passwd
```

Acceso a datos:

```
echo "Primera línea: ${lineas[0]}"  
echo "Total de líneas: ${#lineas[@]}"
```

Desde la salida de un comando

Ejemplo de comando:

```
awk -F':' '{print $1}' /etc/passwd
```

Cargar la salida en un array usando sustitución de procesos:

```
mapfile -t sysusers < <(awk -F':' '{print $1}' /etc/passwd)
```

Consultar el resultado:

```
echo "${#sysusers[@]}"  
echo "${sysusers[45]}"
```

Resumen

- Los arrays indexados permiten agrupar datos relacionados
- Los índices comienzan en `0` y no tienen por qué ser consecutivos
- Son fácilmente iterables con bucles `for`
- `readarray` / `mapfile` facilitan cargar datos externos
- Son fundamentales para scripting estructurado en Bash

04.3 Arrays asociativos

Un **array asociativo** es una colección de pares **clave** → **valor**.

A diferencia de los arrays indexados, las claves **no son números**, sino **cadenas de texto**.

Conceptualmente, un array asociativo es un **diccionario**.

Declaración

Para usar arrays asociativos es obligatorio declararlos explícitamente:

```
declare -A fichaEmpleado
```

Inicialización

Asignación directa por claves

```
fichaEmpleado[Nombre]="John"  
fichaEmpleado[Apellido]="Doe"  
fichaEmpleado[ID]="1234"  
fichaEmpleado[Puesto]="Técnico informático"
```

Inicialización literal

```
declare -A fichaEmpleado=(  
  [Nombre]="John"  
  [Apellido]="Doe"  
  [ID]="1234"  
  [Puesto]="Técnico informático"  
)
```

Acceso a valores

Acceso a un valor concreto mediante su clave:

```
echo "El id del empleado es ${fichaEmpleado["ID"]}"
```

Iteración sobre un array asociativo

Iterar solo sobre los valores

```
for dato in "${fichaEmpleado[@]}; do
    echo "$dato"
done
```

Iterar sobre claves y valores

```
for propiedad in "${!fichaEmpleado[@]}; do
    echo "$propiedad -> ${fichaEmpleado[$propiedad]}"
done
```

Notas:

- `${fichaEmpleado[@]}` → valores
- `${!fichaEmpleado[@]}` → claves

Comprobar existencia de una clave

Para comprobar si una clave existe se usa la opción `-v` con `[[ ... ]]`:

```
[[ -v fichaEmpleado[Nombre] ]] && echo "La ficha tiene nombre"
```

Significado:

- Devuelve verdadero si la clave está definida
- No depende del contenido del valor

Eliminar elementos

Eliminar una clave concreta:

```
unset fichaEmpleado["ID"]
```

Eliminar todo el array:

```
unset fichaEmpleado
```

04.4 Here Documents y Here Strings

Ambos mecanismos redirigen texto hacia la **entrada estándar (stdin)** de un comando.

Here Documents (<<)

Un **Here Document** permite redirigir **varias líneas de texto** como entrada estándar de un comando.

La shell leerá líneas hasta encontrar una que contenga únicamente un **delimitador** definido por el usuario.

Sintaxis general

```
[n]<<word
here-document
word
```

- `word` es el delimitador
- El delimitador **no se expande**
- Debe aparecer solo en una línea para cerrar el bloque

Expansiones en Here Documents

Dentro del here document **sí se permiten**:

- Expansión de variables
- Sustitución de comandos
- Expansión aritmética

Ejemplo:

```
var=1234
cat <<EOF
Valor = $var
Son las $(date +%H:%M)
EOF
```

La shell expande el contenido antes de pasarlo al comando.

Indentación con <<-

Si se usa `<<-` (con guion), Bash permite **tabular el contenido** del here document.

Las **tabulaciones iniciales** se eliminan automáticamente.

```
cat <<-EOF
    Esta línea está tabulada
        Pero es aceptada de igual forma
EOF
```

 Importante:

- Solo se eliminan **tabulaciones**, no espacios

Here Strings (<<<)

Un **Here String** es una versión simplificada del here document, pensada para:

- Una sola línea
- Expresiones cortas

Redirige una cadena directamente a `stdin`.

Sintaxis

```
[n]<<< word
```

`word` admite:

- Expansión de tilde
- Expansión de variables
- Sustitución de comandos
- Expansión aritmética
- Eliminación de comillas

No se realiza:

- Filename expansion (globbing)
- Word splitting

Ejemplo básico

```
comando <<< "texto"
```

Equivalente conceptual a:

```
echo "texto" | comando
```

Pero:

- Más eficiente
- Sin tubería
- Sin subshell
- Sin uso de `echo`

Ejemplo práctico

```
wc -w <<< "Cinco palabras en un string"
```

Comparación conceptual:

```
echo "Cinco palabras en un string" | wc -w
```

El resultado es el mismo, pero el here string es más limpio y directo.

Cuándo usar cada uno

- **Here Document**
 - Texto multilínea
 - Bloques grandes
 - Configuraciones o scripts embebidos
- **Here String**
 - Texto corto
 - Una sola expresión
 - Sustituir `echo | comando`

Resumen

- `<<` permite pasar bloques completos de texto
- `<<<` permite pasar una sola cadena
- Ambos evitan archivos temporales
- Son herramientas clave para scripting limpio y eficiente en Bash

04.5 Named Pipes (FIFOs)

¿Qué es una FIFO?

Una **FIFO (First In, First Out)** es un **fichero especial** que actúa como un **canal de comunicación entre procesos**.

Características clave:

- Funciona como un “tubo” permanente en el sistema de archivos
- Un proceso **escribe** y otro **lee**
- A diferencia de las tuberías (`|`), las FIFOs:
 - Tienen **nombre**
 - **Persisten** hasta que se eliminan

Normalmente se crean en:

- `/tmp`
- El directorio de trabajo

Creación de una FIFO

Para crear una FIFO se usa `mkfifo` :

```
mkfifo nombre_fifo
```

Ejemplo:

```
mkfifo fifo
ls -l fifo
```

Verás que el archivo tiene un tipo especial (`p`).

Uso básico: escritura y lectura

En una terminal (proceso A):

```
echo "Hola desde el proceso A" > canal
```

En otra terminal (proceso B):

```
cat canal
```

Salida:

```
Hola desde el proceso A
```

Bloqueo y sincronización

Las FIFOs **bloquean** por diseño:

- Si se **lee** y nadie escribe → el proceso se bloquea
- Si se **escribe** y nadie lee → el proceso se bloquea

Este comportamiento:

- Garantiza sincronización
- Evita lecturas o escrituras incompletas

Uso en scripts

Las FIFOs permiten:

- Intercambiar datos entre scripts
- Evitar archivos intermedios
- Comunicar procesos que no se ejecutan simultáneamente

Ejemplo conceptual:

```
mkfifo /tmp/f
script1 > /tmp/f &
script2 < /tmp/f
```

Aquí:

- `script1` escribe en background
- `script2` lee desde la FIFO

Manejo del bloqueo

En scripts complejos es habitual:

- Usar procesos en **background**
- Implementar **timeouts**
- Controlar el orden de arranque de procesos

Esto evita bloqueos indefinidos.

Casos de uso habituales

Comunicación entre procesos:

- Pasar datos entre scripts sin archivos intermedios

Sistemas de logging:

- Enviar logs a un proceso que los procesa en tiempo real

Simulación de sockets:

- Comunicación local tipo cliente-servidor

Tuberías persistentes:

- Reemplazar `|` cuando los procesos no coinciden en el tiempo

Procesamiento en streaming:

- Un proceso produce datos continuamente y otro los consume

Ejemplo conceptual:

```
tail -f /var/log/syslog > fifo
```

```
grep error < fifo
```

Limpieza

Las FIFOs **no se eliminan automáticamente**.

Es buena práctica borrarlas cuando ya no se usan:

```
rm fifo
```

Resumen

- Una FIFO es un fichero especial de comunicación
- Permite sincronización natural entre procesos
- Bloquea si no hay lector o escritor
- Es ideal para comunicación persistente y streaming
- Es una herramienta avanzada pero muy potente en Bash

04.6 Subshells y Sustitución de Procesos

Subshells

Aunque ya hemos hablado de subshells en capítulos anteriores, conviene repasar el concepto porque está directamente relacionado con la **sustitución de procesos**.

Una **subshell** es un **proceso independiente**, creado a partir del proceso padre mediante un `fork`.

Características principales:

- Hereda una **copia del entorno** del proceso padre (variables y funciones exportables)
- Ejecuta comandos de forma **aislada**
- Los cambios de variables o directorio **no afectan** a la shell principal

Cuándo se crea una subshell

Hemos usado subshells en varias situaciones:

- Al ejecutar un script como proceso nuevo:

```
./script.sh
```

```
bash script.sh
```

(Lo contrario es hacer *sourcing* con `source` o `.`)

- Al agrupar comandos con paréntesis:

```
( comando1; comando2 )
```

En ambos casos, los cambios realizados **no persisten** fuera de la subshell.

Sustitución de procesos

Es posible que hayas visto esta sintaxis en scripts Bash:

```
<( ... )
```

```
>( ... )
```

La **sustitución de procesos** permite **tratar la entrada o salida de un comando como si fuera un archivo**.

Internamente, Bash:

- Ejecuta el comando en una subshell
- Conecta su entrada o salida a una FIFO o a `/dev/fd/N`

Sintaxis

```
<(comando)
```

Sustituye la **salida estándar (stdout)** del comando por un archivo.

```
>(comando)
```

Sustituye la **entrada estándar (stdin)** del comando por un archivo.

Ejemplo con `<( ... )`

El comando `diff` compara dos archivos línea a línea:

```
diff file1 file2
```

Comparar la salida de dos comandos sin archivos temporales:

```
diff <(ls /bin) <(ls /usr/bin)
```

Qué ocurre internamente:

- `ls /bin` y `ls /usr/bin` se ejecutan en procesos separados
- Cada salida se conecta a un descriptor tipo `/dev/fd/X`
- `diff` recibe dos “archivos”:

```
diff /dev/fd/63 /dev/fd/62
```

Ejemplo con `>( ... )`

Aquí el comando dentro de `>( ... )` **recibe la entrada por stdin**.

Ejemplo de procesamiento múltiple de una misma salida:

```
man man | tee >(grep "manual") >(wc -l) >(wc -c)
```

En este caso:

- `tee` duplica la salida
- Cada sustitución de proceso recibe una copia
- Cada comando procesa la entrada de forma independiente

Ejemplo sobre la entrada estándar

Duplicar la entrada del terminal y clasificarla:

```
tee >(grep "ERROR" > errores.log) >(grep "INFO" > info.log)
```

Aquí:

- `tee` lee de stdin
- Envía la entrada a dos procesos distintos
- Cada uno filtra y escribe en su fichero

Relación con subshells

La sustitución de procesos:

- Usa **subshells internamente**
- Ejecuta comandos en procesos separados
- No modifica el entorno del proceso padre

Portabilidad

La sustitución de procesos:

- Es una **expansión avanzada de la shell**
- No está disponible en shells antiguas o estrictamente POSIX
- Puede fallar en `sh` o `dash`

Si la **portabilidad** es importante, conviene evitarla.

Resumen

- Una subshell es un proceso aislado
- `( ... )` crea subshells explícitamente
- `<( ... )` y `>( ... )` permiten tratar comandos como archivos
- Evitan archivos temporales
- Son muy potentes, pero menos portables

04.7 Descriptores de ficheros (FD)

Un **descriptor de fichero (FD)** representa un **recurso abierto**:

- Ficheros
- Pipes
- Dispositivos
- Sockets

Es un **número entero** que el sistema operativo asigna a cada recurso de **entrada/salida (E/S)** que un proceso tiene abierto.

Cada proceso mantiene internamente una **tabla de descriptores**, que apunta a los recursos disponibles.

Descriptores estándar

En Bash (y Unix en general) existen tres descriptores creados por defecto:

- 0 → **stdin** (entrada estándar)
- 1 → **stdout** (salida estándar)
- 2 → **stderr** (salida de errores)

Ejemplos habituales:

- `stdin` → teclado o pipe
- `stdout` → pantalla o fichero
- `stderr` → pantalla o fichero de logs

Piensa en los FDs como **canales numerados** por los que fluye la información.

Ver descriptores activos

En `/dev` :

```
ls -l /dev/std*
```

En `/proc` para el proceso actual:

```
ls /proc/$$/fd/
```

Nota:

- En Bash, el FD 255 se usa internamente para mantener una referencia al terminal (TTY), incluso si 0, 1 y 2 han sido redirigidos.

Redirecciones (uso implícito de FDs)

Ya has usado FDs indirectamente al redirigir salida o errores:

```
./script.sh > /tmp/file.txt
```

Equivale conceptualmente a:

```
./script.sh 1> /tmp/file.txt
```

Redirección de errores:

```
./script.sh 2> /tmp/errors.txt
```

Redirección conjunta:

```
./script.sh &> /tmp/script.log
```

Descriptores personalizados

Además de 0, 1 y 2, Bash permite **abrir nuevos descriptores** con el builtin `exec`.

Ejemplo:

```
exec 3>log_ok.txt
exec 4>log_err.txt

echo "Inicio del proceso" >&3
ls /noexiste >&4
echo "Fin del proceso" >&3
```

```
exec 3>&-
exec 4>&-
```

Esto permite:

- Mantener múltiples flujos abiertos
- Separar logs por tipo
- Evitar abrir/cerrar ficheros repetidamente

 Usar FDs mayores que 9 con cuidado en shells interactivas (No recomendable, están reservados)

Operaciones avanzadas con FDs

Bash permite **redirigir, duplicar, agrupar y cerrar** descriptores.

Redirigir FDs con exec

```
exec 1>output.log 2>error.log
```

A partir de ese punto:

- `stdout` va a `output.log`
- `stderr` va a `error.log`

Duplicar FDs

Sintaxis:

```
[n]>&m
```

Ejemplo clásico:

```
./script.sh >file.log 2>&1
```

Ejemplo con FD personalizado:

```
exec 3>todo.log
echo "stdout normal" >&3
ls /noexiste >&3 2>&1
exec 3>&-
```

Agrupación de comandos con redirecciones

Aplicar redirecciones a un bloque completo:

```
{
  echo "Inicio del proceso"
  date
  ls /noexiste
} >salida.log 2>&1
```

Funciona tanto con `{ ...; }` como con `( ... )` (recordando la diferencia de subshell).

Cerrar descriptores

Para cerrar un FD:

```
exec 3>&-
```

Una vez cerrado:

- `>&3` ya no es válido

Resumen

- Los FDs son números que representan recursos abiertos
- `0`, `1` y `2` son `stdin`, `stdout` y `stderr`
- `exec` permite abrir, redirigir y cerrar FDs
- Los FDs personalizados permiten control fino de E/S
- Son fundamentales para logging avanzado y scripting robusto

04.8 Procesos en background

Comprender cómo manejar **procesos en segundo plano** desde un script Bash, cómo **sincronizarlos**, y cómo **comprobar su finalización**, incluso cuando se lanzan múltiples tareas simultáneamente.

Job control (uso interactivo)

Bash dispone de comandos pensados para **uso interactivo**, no para scripts:

- `jobs` → lista trabajos en segundo plano
- `fg` → trae un trabajo al primer plano
- `bg` → continúa un trabajo en segundo plano

Ejemplo en una shell interactiva:

```
ping -c 3 google.com
```

```
ping -c 5 google.com >/dev/null &  
jobs
```

```
ping -c 15 google.com >/dev/null &  
jobs
```

```
fg %1
```

Esto se conoce como **job control**.

 Importante:

- `fg`, `bg` y `jobs` **no se usan en scripts**
- Están pensados para shells interactivas

Procesos en background en scripts

En scripts, el control se hace de otra forma.

Para ejecutar un comando en segundo plano se usa `&`:

comando `&`

Esto:

- Lanza el proceso en background
- Devuelve inmediatamente el control al script

Obtener el PID de un proceso

El PID del **último proceso lanzado en background** se almacena en:

```
$!
```

Ejemplo:

```
sleep 5 &  
pid=$!  
echo "PID del proceso: $pid"
```

El builtin wait

El comando integrado `wait` permite:

- Esperar a que termine un proceso
- Obtener su estado de salida

Esperar a un PID concreto:

```
wait PID
```

Esperar a **todos** los procesos en background:

```
wait
```

El exit status de `wait` es:

- El del proceso esperado
- O el último proceso si no se especifica PID

Ejemplo: tareas en paralelo

Ejecutar varias tareas simultáneamente y esperar a que todas terminen:

```
sleep 2 &
pid1=$!

sleep 4 &
pid2=$!

sleep 6 &
pid3=$!

wait $pid1
wait $pid2
wait $pid3

echo "Todas las tareas han terminado"
```

Aquí:

- Las tareas se ejecutan en paralelo
- El script se sincroniza usando `wait`

Esperar y comprobar errores

Podemos comprobar si un proceso falló:

```
comando &
pid=$!

wait $pid
status=$?

if [[ $status -ne 0 ]]; then
    echo "El proceso falló"
fi
```

Esto es clave para:

- Scripts robustos
 - Control de errores en paralelo
-

Caso típico: lanzar muchos procesos

Ejemplo conceptual:

```
for f in *.log; do
    procesar "$f" &
done

wait
echo "Procesamiento completo"
```

Todos los procesos se lanzan en background y el script espera a que finalicen.

Resumen

- `&` lanza procesos en segundo plano
- `$!` guarda el PID del último proceso
- `wait` sincroniza procesos
- `fg`, `bg` y `jobs` son solo para uso interactivo
- `wait` es la herramienta correcta para scripts
- Permite paralelismo real y controlado en Bash

04.9 Señales

Qué es una señal

Una **señal** es un mensaje que el sistema operativo envía a un proceso para notificarle un evento.

Ejemplos comunes:

Señal	Nombre	Descripción
2	SIGINT	Interrupción desde teclado (Ctrl+C)
15	SIGTERM	Petición de terminación “educada”
1	SIGHUP	Cierre de sesión / terminal
9	SIGKILL	Terminación forzada (no atrapable)
14	SIGALRM	Alarma / timeout

Para ver la lista completa:

```
trap -l
```

```
kill -l
```

```
man 7 signal
```

El builtin trap

El comando integrado **trap** permite **asociar comandos o funciones** a una o varias señales.

Sintaxis básica:

```
trap 'comandos' SEÑAL
```

O usando una función (forma recomendada):

```
trap nombre_funcion SEÑAL
```

Ejemplo básico: capturar Ctrl+C (SIGINT)

```
cleanup() {  
    echo "Interrupción detectada. Limpiando recursos..."  
    rm -f /tmp/archivo_temporal  
    exit 1  
}  
  
trap cleanup SIGINT  
  
echo "Script en ejecución. Pulsa Ctrl+C para interrumpir."  
while true; do  
    sleep 1  
done
```

Cuando el usuario pulsa `Ctrl+C`:

- Bash recibe `SIGINT`
- Se ejecuta `cleanup`
- El script sale limpiamente

Capturar múltiples señales

```
trap cleanup SIGINT SIGTERM SIGHUP
```

Esto es habitual en scripts que:

- Se ejecutan en background
- Son gestionados por `systemd`, `cron`, etc.

Señales que NO pueden atraparse

Hay señales que **no pueden ser capturadas ni ignoradas**:

- `SIGKILL` (9)

- SIGSTOP

Esto es una decisión del kernel.

Uso típico en scripts reales

- Borrar ficheros temporales
 - Cerrar descriptores de fichero
 - Terminar procesos hijos
 - Restaurar configuraciones
 - Registrar el motivo de salida
-

Resumen

- Las señales son eventos enviados por el sistema a procesos
- `trap` permite capturarlas y reaccionar
- `SIGINT` , `SIGTERM` y `SIGHUP` son las más habituales
- `EXIT` garantiza limpieza final
- `SIGKILL` no puede atraparse
- `trap` es clave para scripts **robustos y seguros**

04.10 Coprocesos

¿Qué es un coproceso?

Un **coproceso** es un comando que se ejecuta en **segundo plano**, pero manteniendo **canales abiertos de comunicación** entre el script y ese proceso.

A diferencia de:

- una tubería (|) → comunicación unidireccional
- una FIFO → fichero intermedio

`coproc` permite **entrada y salida simultáneas**.

Sintaxis básica

```
coproc nombre { comando; }
```

Ejemplo simple:

```
coproc CALC { bc; }
```

¿Qué crea Bash internamente?

Al usar `coproc`, Bash crea automáticamente:

- Un **proceso en background**
- Un **pipe para enviarle datos** (stdin del coproceso)
- Un **pipe para leer su salida** (stdout del coproceso)

Todo esto sin que tengamos que gestionar FDs manualmente.

Descriptores asociados al coproceso

El coproceso queda referenciado por un array:

```
nombre[0]
```

→ Descriptor para **leer** (stdout del coproceso)

```
nombre[1]
```

→ Descriptor para **escribir** (stdin del coproceso)

Ejemplo básico de comunicación

```
coproc CALC { bc; }

echo "2 + 3" >&${CALC[1]}"
read resultado <&${CALC[0]}"

echo "Resultado: $resultado"
```

Aquí:

- Escribimos una expresión a `bc`
- Leemos el resultado desde su salida
- El proceso sigue vivo

Cierre de descriptores

Es **muy importante** cerrar los descriptores cuando ya no se usan:

```
exec {CALC[1]}>&-
exec {CALC[0]}<&-
```

Si no se cierran:

- El coproceso puede quedar esperando
- El script puede bloquearse

Uso típico: proceso de larga vida

Los coprocesos son ideales para comandos que:

- Mantienen estado
- Procesan múltiples entradas
- No queremos relanzar cada vez

Ejemplos habituales:

- `bc`
 - `grep`
 - `gpg`
 - `ping`
 - intérpretes interactivos
-

Ventajas frente a otras técnicas

- Comunicación **bidireccional**
 - Sin archivos temporales
 - Sin FIFOs persistentes
 - Más eficiente que múltiples pipes
 - Control directo mediante FDs
-

Limitaciones

- Es una característica **específica de Bash**
 - No es POSIX
 - No funciona en shells antiguas (`sh` , `dash`)
-

Resumen

- `coproc` lanza procesos en paralelo
- Mantiene comunicación bidireccional
- Usa descriptors internos (`[0]` y `[1]`)
- Es ideal para procesos interactivos y de larga duración
- Requiere cerrar correctamente los FDs

04.11 Internal Field Separator (IFS)

Qué es IFS

IFS es una **variable especial de Bash** que define **cómo se separan los campos** cuando Bash divide texto en partes (campos).

Un **campo** es cada fragmento resultante tras dividir una cadena usando los caracteres definidos en IFS .

Por defecto, IFS contiene:

- espacio
- tabulación
- salto de línea

Contenido por defecto de IFS

```
echo "$IFS" | od -c
```

```
printf '%q\n' "$IFS"
```

Ejemplo básico de word splitting

```
texto="uno dos tres:cuatro"  
echo $texto
```

Resultado conceptual:

- Bash ve **tres campos**: uno , dos , tres:cuatro

Modificando IFS

```
IFS=:  
echo $texto
```

Resultado conceptual:

- **Dos campos:** uno dos tres y cuatro

⚠ Cambiar IFS cambia cómo Bash divide texto.

Cuándo se usa IFS

IFS interviene **después de las expansiones**, en los siguientes casos principales.

1. Expansión de variables sin comillas

```
IFS=:  
echo $texto
```

Proceso:

1. Expande la variable
2. Aplica word splitting usando IFS
3. Ejecuta el comando

Con comillas dobles:

```
echo "$texto"
```

- Expande la variable
- **No realiza word splitting**
- Se mantiene un solo campo

2. Comando integrado read

read usa IFS para separar campos leídos desde stdin.

```
read -r a b c  
aaaa bbbb cccc  
  
echo -e "a = $a\nb = $b\nc = $c"  
#salida  
a = aaaa
```



```
b = bbbb
c = cccc
```

Con IFS modificado:

```
IFS=:
read -r a b c
aaaa:bbbb:cccc

echo -e "a = $a\nb = $b\nc = $c"
#salida
a = aaaa
b = bbbb
c = cccc
```

Campos mixtos:

```
read -r a b c
aaaaa bbbbb:cccc

echo -e "a = $a\nb = $b\nc = $c"
#salida
a = aaaaa
b = bbbbb:cccc
c =
```

Exceso de campos:

```
read -r a b c
aaaaa:bbbb:cccc:dddd:eeee

echo -e "a = $a\nb = $b\nc = $c"
#salida
a = aaaaa
b = bbbbb
c = cccc:dddd:eeee
```

3. Expansión de \$*

`$*` une todos los parámetros posicionales usando el **primer carácter de IFS**.

Ejemplo:

```
set -- x y "z zz"
echo $*
x y z zz #salida
```

Con IFS modificado:

```
IFS=:
echo $*
x y z zz #salida
```

Con comillas:

```
echo "$*"
x:y:z zz #salida
```

Resultado conceptual:

- x:y:z zz

⚠ Nunca asumas que \$* preserve los parámetros originales:

- Depende de IFS
- Puede colapsar argumentos
- Se comporta distinto a \$@

4. Sustitución de comandos

```
IFS=:
for x in $(echo "a:b:c"); do
    echo "$x"
done

#salida
a
b
c
```

La salida del comando:

- Se expande
- Se divide en campos usando IFS

5. Expansión de arrays con [*]

```
IFS=,
arr=(a b c)
echo "${arr[*]}"
a,b,c #salida
```

Resultado:

- a,b,c

Nota:

- "\${arr[*]}" **sí usa IFS**
- "\${arr[@]}" **no usa IFS** en este contexto

6. Bucles for y while

```
IFS=\
lista="a:b:c d:e"
for x in $lista; do
    echo "$x"
done

#salida
[a:b:c]
[d:e]
```

Cambiando IFS:

```
IFS=:

#salida
[a]
[b]
```

```
[c d]
[e]
```

Bash divide según el separador activo.

Lectura segura con while

Contenido de ejemplo "/tmp/datos.txt":

```
uno dos tres
cuatro:cinco:seis
  línea con espacios al inicio
línea\ con\ barras
```

Lectura insegura:

```
while read line; do
  echo "[$line]"
done < /tmp/datos.txt

#IFS por defecto
#salida
[uno dos tres]
[cuatro:cinco:seis]
[línea con espacios al inicio]
[línea con barras]
```

Lectura segura (recomendada):

```
while IFS= read -r line; do
  echo "[$line]"
done < /tmp/datos.txt

#salida
[uno dos tres]
[cuatro:cinco:seis]
[ línea con espacios al inicio]
[línea\ con\ barras]
```

Comparación:

```
diff <(while read line; do echo "$line"; done < /tmp/datos.txt) \
    <(while IFS= read -r line; do echo "$line"; done < /tmp/datos.txt)

3,4c3,4
< [línea con espacios al inicio]
< [línea con barras]
---
> [  línea con espacios al inicio]
> [línea\ con\ barras]
```

Conclusión

- IFS define **cómo Bash divide texto en campos**
- **Solo se aplica después de las expansiones**
- Interviene en:
 - word splitting
 - read
 - `$*` y `"$*"`
 - `$(...)`
 - `"${array[*]}"`
- Las **comillas dobles desactivan el word splitting**, salvo en `$*` y `${array[*]}`
- `IFS=` desactiva la división en campos (patrón seguro con `read`)
- `$*` **siempre depende de IFS**
- `"$@"` es la forma **segura y correcta** de reenviar parámetros

Regla de oro

Entrecomilla siempre las variables

y modifica IFS **solo de forma local y consciente**